



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

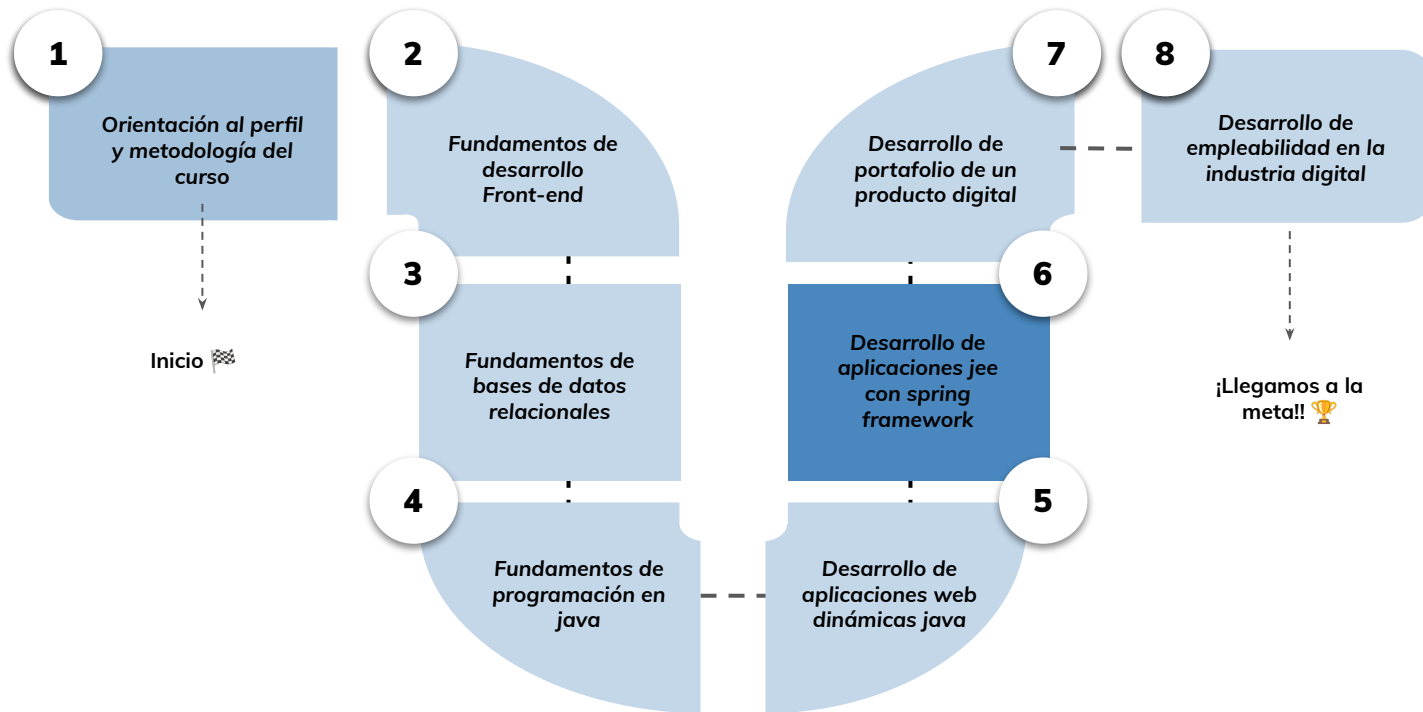


› El framework spring MVC - Parte I

Aprendizaje Esperado 2: Utilizar framework spring boot y spring mvc para la implementación de una aplicación web básica de acuerdo a las buenas prácticas de la industria.

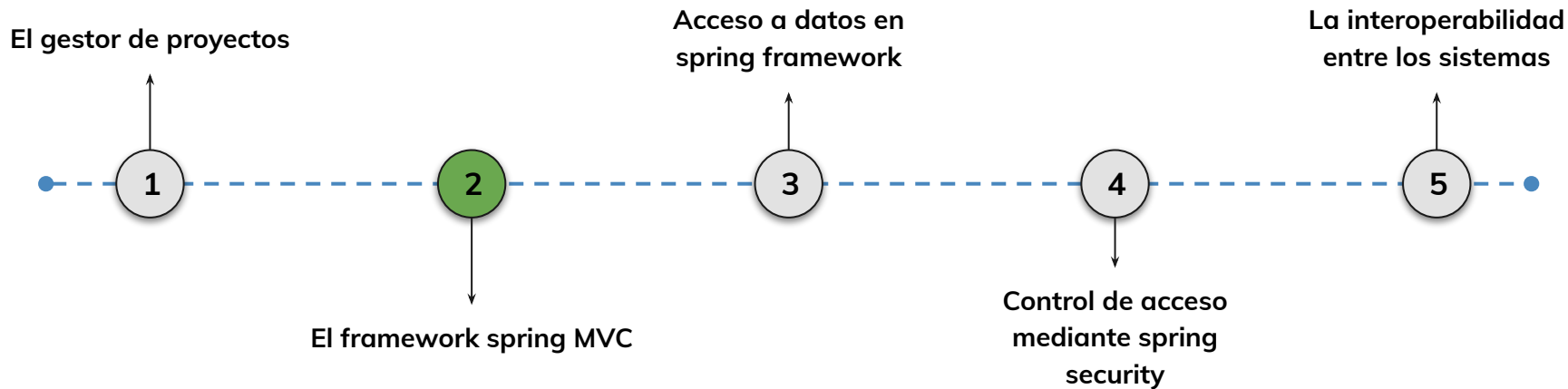
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



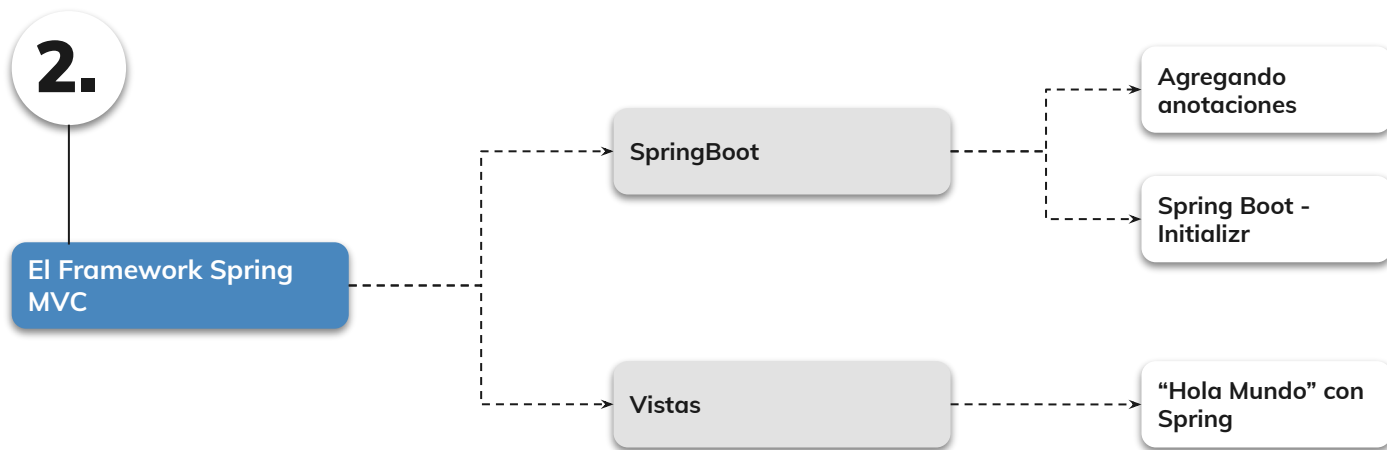
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?



Objetivos de aprendizaje

¿Qué aprenderás?

- Conocer las características de Spring Boot
- Aprender conceptos de anotaciones y Bean
- Comprender el concepto de inyección de dependencias
- Conocer las características de Spring Initializr
- Conocer las configuraciones para tecnologías de vistas
- Comprender la configuración de un datasource

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior :

- Aprendimos el concepto de ciclo de vida en proyectos Java.
- Conocimos las etapas del ciclo de compilación.
- Aprendimos a instalar y ejecutar proyectos utilizando Maven.
- Conocimos el proceso de empaquetamiento de aplicaciones con Maven.

» Spring Boot

Spring Boot

Spring Boot es un popular y poderoso framework de desarrollo de aplicaciones Java que simplifica considerablemente la creación de aplicaciones basadas en Spring Framework. Ofrece una amplia gama de características y beneficios que lo convierten en una elección atractiva para desarrolladores y equipos de desarrollo de todo el mundo.



Spring Boot[®]

Spring Boot

Características principales :

1. **Simplifica la configuración** : Spring Boot ofrece una configuración por defecto que es bastante sensata, lo que significa que puedes comenzar a desarrollar sin tener que preocuparte por detalles de configuración complicados. Además, proporciona anotaciones y propiedades que permiten personalizar la configuración según tus necesidades.
2. **Incorpora un servidor de aplicaciones embebido** : Spring Boot incluye servidores de aplicaciones embebidos, como Tomcat, Jetty o Undertow. Esto significa que no es necesario desplegar tu aplicación en un servidor externo; puedes ejecutarla directamente como una aplicación independiente.

Spring Boot

Características principales :

3. Gestión de dependencias : Spring Boot incorpora una gestión de dependencias sólida a través de Apache Maven o Gradle. Esta característica permite especificar las dependencias necesarias en el archivo pom.xml o build.gradle, y Spring Boot se encarga de descargar automáticamente las bibliotecas correctas y gestionar las versiones de dependencias.

4. Creación de aplicaciones autónomas : Lo que significa que una aplicación puede funcionar de manera independiente sin requerir configuraciones externas complicadas. Esto facilita la implementación y la ejecución de aplicaciones en diferentes entornos, como servidores locales, contenedores o plataformas en la nube.

Spring Boot

Características principales :

5. Autoconfiguración inteligente : Significa que ajusta automáticamente la configuración de la aplicación en función de las bibliotecas y dependencias que detecta en el proyecto. Esto reduce la necesidad de configuraciones manuales y garantiza que la aplicación esté configurada de manera óptima.

6. Desarrollo basado en anotaciones : Las anotaciones como @Controller, @Service, @Repository, @RequestMapping, entre otras, simplifican el desarrollo de aplicaciones web y permiten una estructura clara y modular.

Spring Boot

Características principales :

7. Integración de bases de datos: Proporcionando herramientas como Spring Data JPA y Spring Data MongoDB, que facilitan la implementación de la capa de persistencia. Además, ofrece soporte para bases de datos en memoria para pruebas y desarrollo.

8. Soporte para pruebas unitarias: Se integra perfectamente con frameworks de pruebas como JUnit y TestNG. Esto facilita la escritura y ejecución de pruebas unitarias y de integración, lo que garantiza la calidad del código y la funcionalidad de la aplicación.

Spring Boot

Características principales :

9. Actuadores y monitoreo: Spring Boot proporciona una serie de "actuadores" que permiten supervisar y administrar la aplicación en tiempo real. Puedes obtener información sobre métricas, salud de la aplicación, estado de las conexiones de base de datos y más.

10. Compatibilidad con Spring Ecosystem: Lo que te permite aprovechar las ventajas de las características y bibliotecas de Spring existentes. Esto incluye Spring Data, Spring Security, Spring Cloud y muchos otros proyectos de Spring.

Spring Boot

Características principales :

11. Desarrollo ágil y rápido prototipado: La combinación de autoconfiguración, servidores de aplicaciones embebidos y una estructura de proyectos clara permite a los desarrolladores centrarse en la lógica de negocio en lugar de preocuparse por la configuración y la infraestructura.

12. Soporte para aplicaciones basadas en microservicios: Spring Boot es una elección popular para desarrollar aplicaciones basadas en microservicios debido a su capacidad para crear aplicaciones autónomas y su integración con Spring Cloud, que proporciona herramientas para la gestión de microservicios.

Spring Boot

Características principales :

13. Comunidad activa y amplia documentación: Esto facilita la obtención de ayuda, la solución de problemas y el aprendizaje de nuevas características y mejores prácticas.

14. Implementación en la nube: Se integra con facilidad en plataformas en la nube como AWS, Google Cloud, Microsoft Azure y Heroku. Esto permite la implementación de aplicaciones en entornos en la nube de manera sencilla y eficiente.

15. Seguridad: Spring Boot ofrece funcionalidades de seguridad, como autenticación y autorización, que se pueden configurar de manera sencilla para proteger las aplicaciones web. También se integra con Spring Security, un proyecto de Spring ampliamente utilizado para la gestión de la seguridad.

➤ Anotaciones

Anotaciones

Las anotaciones proporcionan una forma de agregar metadatos enriquecidos a los elementos del código. Esto permite a los desarrolladores adjuntar información adicional a clases, métodos o campos, como autor, fecha de creación, descripción, restricciones de seguridad y más. Estos metadatos pueden ser utilizados por herramientas de desarrollo, generación de documentación y otras tareas que requieren información contextual sobre el código.

Anotaciones

Personalización del comportamiento:

Las anotaciones se utilizan para personalizar el comportamiento de clases, métodos y campos. Esto se logra mediante la configuración de anotaciones específicas que indican cómo una clase o método debe comportarse en tiempo de ejecución. Por ejemplo, las anotaciones como **@Override** o **@Deprecated** influyen en cómo el compilador y otras herramientas tratan el código.

Validación y restricciones:

Las anotaciones se emplean para aplicar restricciones y validaciones en los datos de entrada y los objetos. Por ejemplo, las anotaciones de validación como **@NotNull**, **@Min**, **@Max** y **@Pattern** permiten definir restricciones en los campos de una entidad, lo que garantiza que los datos cumplan con ciertos criterios antes de ser procesados.

Anotaciones

Extensibilidad y creación de anotaciones personalizadas:

Java permite a los desarrolladores crear sus propias anotaciones personalizadas. Esto brinda una flexibilidad adicional para definir metadatos y comportamientos específicos de la aplicación. La capacidad de crear anotaciones personalizadas ha llevado a la creación de numerosas bibliotecas y marcos que hacen un uso extensivo de anotaciones adaptadas a casos de uso específicos.

Anotaciones

Compatibilidad con anotaciones de Java SE y EE :

Java SE y EE proporcionan un conjunto de anotaciones predefinidas que son ampliamente utilizadas en el desarrollo de aplicaciones empresariales y aplicaciones de escritorio. Estas anotaciones incluyen **@Override** , **@Deprecated** , **@Entity** , **@Resource** , **@WebService** y muchas más. La compatibilidad con estas anotaciones está integrada en el lenguaje Java y en su infraestructura de desarrollo.

Anotaciones

Compatibilidad con anotaciones de Java SE y EE :

Java SE y EE proporcionan un conjunto de anotaciones predefinidas que son ampliamente utilizadas en el desarrollo de aplicaciones empresariales y aplicaciones de escritorio. Estas anotaciones incluyen **@Override** , **@Deprecated** , **@Entity** , **@Resource** , **@WebService** y muchas más. La compatibilidad con estas anotaciones está integrada en el lenguaje Java y en su infraestructura de desarrollo.

Bean

¿Qué es un Bean?

Un Bean es una clase de Java que debe cumplir los siguientes requisitos:

- Tener todos sus atributos privados (private).
- Tener métodos set() y get() públicos de los atributos privados que nos interese.
- Tener un constructor público por defecto.

A diferencia de los Bean convencionales que representan una clase, la particularidad de **los Beans de Spring** es que **son objetos creados y manejados por el contenedor Spring**.

Bean

Anotación: @Configuration

@Configuration

Se utiliza para la configuración basada en anotaciones. Indica que una clase declara uno o más beans y será procesada por el contenedor Spring.

Por lo general, recomendamos utilizar @Configuration en una clase exclusiva de configuraciones.

Ejemplo

```
@Configuration
public class AppConfig {
    // ... aquí se definirán los beans
}
```


Bean

Anotación: @Bean

@Bean

Se utiliza a nivel de método. Funciona con

@Configuration para crear beans Spring.

El método anotado con esta anotación funciona como la ID del bean, y crea y devuelve el bean real.

Ejemplo

```
@Configuration
public class AppConfig {

    @Bean
    public Person person() {
        return new Person(address());
    }

    @Bean
    public Address address() {
        return new Address();
    }
}
```

Bean

Anotación: @Autowired

@Autowired

Le indica a Spring dónde debe ocurrir una **inyección de dependencia**.

Spring busca un bean del tipo requerido y, si lo encuentra, lo inyecta. Sustituye la declaración de atributos en XML.

Se emplea para la inyección en clases con @Component (o @Controller, @Service, @Repository).

Ejemplo

```
@Service
public class MyService {

    // Spring inyectará una instancia
    // de AddressService automáticamente
    @Autowired
    private final AddressService addressService;

    // ...
}
```

Bean

Anotación: @Component

@Component

Le indica a Spring Boot que debe tratar la clase con el decorador como un bean, es decir que el propio motor se encargará de hacer la inyección.

Es la anotación genérica para cualquier componente gestionado por Spring.

Ejemplo

```
@Component
public class MyClass {
    // ...
}
```

Bean

Capas de Aplicación

@Repository

Se utiliza para la **capa de persistencia**, en otras palabras, clases que interactúan con la base de datos. Tiene un control de errores más específico.

@Service

Se utiliza en la **capa de servicio**, es decir, en la clase que se espera que sea la que maneje la lógica de negocio.

@Controller

Nos enlaza con la **capa de presentación** (vistas). Allí solemos colocar las URLs que se exponen vía web.

LIVE CODING

Overview de un proyecto Spring Boot:

En esta primera parada, veremos la estructura de un proyecto Spring Boot básico con las anotaciones iniciales que ayudan a su funcionamiento.

Tiempo: 10 minutos

➤ Creando un proyecto Spring Boot

Creando un proyecto Spring Boot

Al igual que en cualquier biblioteca Java estándar, en Spring Boot se incluyen los correspondientes archivos JAR (Java Archive) o WAR (Web Application Archive) en el Classpath. Java recurre a esta ruta del sistema para buscar los archivos ejecutables. Puedes crear los archivos de almacenamiento de Spring Boot de dos maneras distintas:

- **Instala y utiliza Maven** para crear por tu cuenta todo el marco del proyecto, incluidas las dependencias necesarias.
- **Utiliza el servicio web Spring Initializr** para establecer la configuración de Spring Boot y, después, descárgala como plantilla de proyecto final.

Creando un proyecto Spring Boot

Una característica de **Spring Initializr** es que proporciona una interfaz web muy fácil de usar para crear los archivos JAR, lo que simplifica considerablemente el proceso. Como **Initializr también emplea Maven** para generar los archivos, el resultado es el mismo que si lo haces de forma manual.



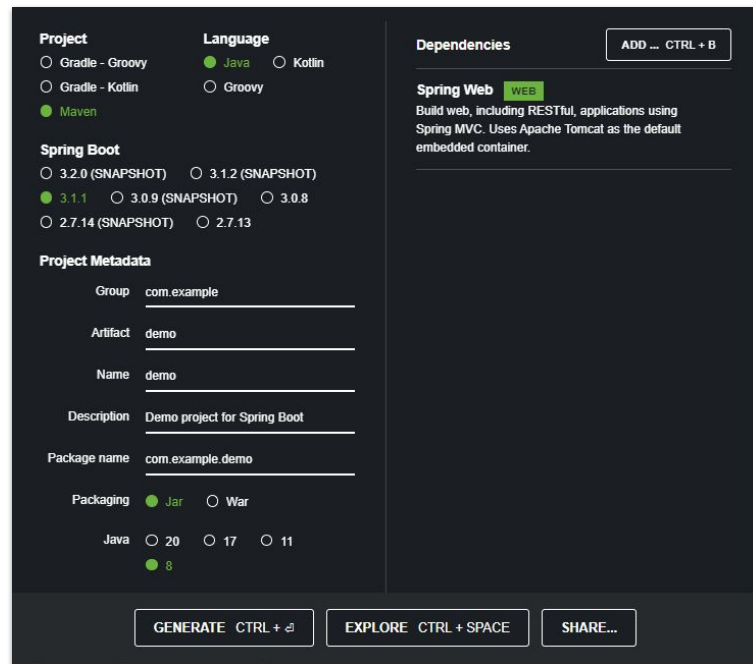
Creando un proyecto Spring Boot

Para iniciar un proyecto Spring Boot :

- 1- Ingresar al sitio web Spring Initializr en start.spring.io .
- 2- Configurar el proyecto, incluyendo las dependencias (en este caso comenzaremos con la dependencia Spring Web).

» Configurando un proyecto Spring Boot

Configurando un proyecto Spring Boot



The screenshot shows the Spring Boot project configuration interface. It is divided into three main sections: Project, Language, and Dependencies.

Project

- ☐ Gradle - Groovy
- ☐ Gradle - Kotlin
- ☒ Maven

Language

- ☒ Java
- ☐ Kotlin
- ☐ Groovy

Spring Boot

- ☐ 3.2.0 (SNAPSHOT)
- ☐ 3.1.2 (SNAPSHOT)
- ☒ 3.1.1
- ☐ 3.0.9 (SNAPSHOT)
- ☐ 3.0.8
- ☐ 2.7.14 (SNAPSHOT)
- ☐ 2.7.13

Project Metadata

Group:

Artifact:

Name:

Description:

Package name:

Packaging: ☒ Jar ☐ War

Java: ☐ 20 ☐ 17 ☐ 11

☒ 8

Dependencies ADD ... CTRL + B

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Buttons: GENERATE CTRL + G, EXPLORE CTRL + SPACE, SHARE...

Configurando un proyecto Spring Boot

Configuraciones:

Project : Permite elegir la herramienta de construcción de la aplicación. En Java las dos herramientas más usadas son Maven y Gradle. Recomendamos Maven al ser la más extendida.

Language : Lenguaje de programación que se va a utilizar en la aplicación. Los tres tipos están soportados por la máquina virtual JVM. Java es la opción más extendida y tiene mejor soporte de los editores de programación.

Spring Boot : Versión del Spring Boot a usar. Siempre que se pueda se optará por la última estable, compuesta únicamente por números (sin palabras entre paréntesis).

Configurando un proyecto Spring Boot

Project Metadata :

- **Group**: Utilizado para clasificar el proyecto en los repositorios de binarios. Normalmente se suele usar una referencia similar a la de los packages de las clases. Por ejemplo, com.example para disponer todas las aplicaciones web en el mismo directorio.
- **Artifact** : Para indicar el nombre del proyecto y del binario resultante. La combinación de groupId y artifactId (más la versión) son identificadores únicos.
- **Name**: Asignar un nombre al proyecto.
- **Description** : Determina una descripción al proyecto a crear.
- **Packaging name** : Se refiere al nombre principal del paquete de nuestra aplicación.

Configurando un proyecto Spring Boot

Project Metadata :

- **Packaging** : Indica qué tipo de binario se debe construir. Si la aplicación se ejecutará por sí sola se seleccionará JAR, éste contiene todas las dependencias dentro de él y se podrá ejecutar con `java -jar binario-.jar`. Si por el contrario, la aplicación se ejecutará en un servidor J2EE existente o en un Tomcat ya desplegado se deberá escoger WAR.
- **Java** : Se selecciona la versión de Java a usar. En este caso, se recomienda usar la versión **estable** de Java más antigua para garantizar la compatibilidad con otras librerías o proyectos que se quieran incluir, así será más probable encontrar documentación existente que siga siendo válida. Reduce el riesgo de toparse con funcionalidades no muy maduras.

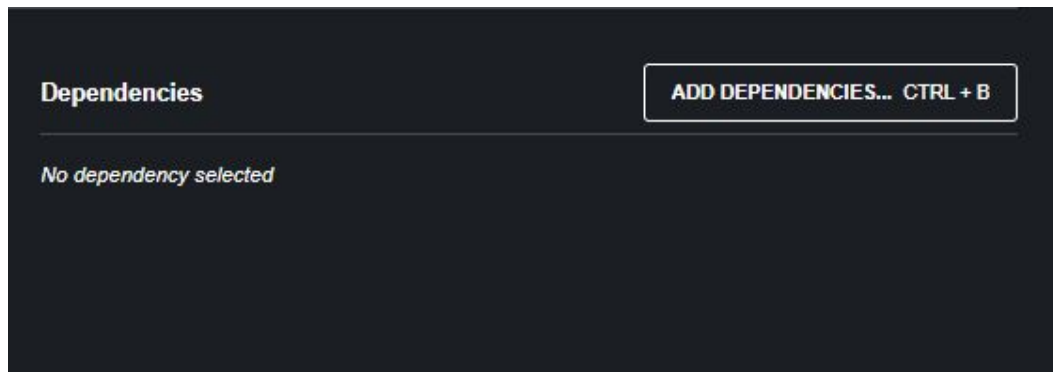
Configurando un proyecto Spring Boot

Del lado derecho de la pantalla podremos encontrar el botón de buscador de dependencias **Add Dependencies** : Buscador de dependencias con los starters de Spring boot disponibles. Las dependencias más habituales son:

- **Spring Web:** Orientado al desarrollo de aplicación web o microservicios.
- **Thymeleaf** : Incorpora el motor de plantillas para HTML dinámico (sucesor de JSP - Java Server Page).
- **Spring Data JPA** : Capa estándar de acceso a base de datos SQL denominada Java Persistence Api.
- **Spring Security** : Incorporar controles de acceso en base a usuarios y roles sobre URLs de la aplicación. Asimismo, habilita el control de ejecución de métodos de servicio en base a roles según los estándares J2EE.

Configurando un proyecto Spring Boot

- **Lombok**: Facilita la programación como la creación de @Getters y @Setters automáticamente para las clases que forman parte del conjunto de mensajes.
- **Mysql/Postgresql**: Incluye el JAR que contiene el driver JDBC necesario para configurar la capa de JPA según la base de datos a usar.



Configurando un proyecto Spring Boot

Después de seleccionar los parámetros necesarios se requiere generar el proyecto, para lo cual se tiene que hacer click en el botón "Generate Ctrl+" y automáticamente se descargará el archivo zip con el nombre del Artifact que contendrá la carpeta con la estructura de la aplicación lista para importar desde el IDE de tu preferencia.

GENERATE CTRL + ↵

EXPLORE CTRL + SPACE

SHARE...

Configurando un proyecto Spring Boot

Después de la descarga del archivo, se debe descomprimir el archivo ZIP con la finalidad de importar o abrir el proyecto creado.

Una vez extraído el archivo, ya podemos abrirlo en Eclipse IDE, seleccionando la pestaña File, luego Open Projects from File System y a continuación eligiendo la carpeta del directorio que contiene el proyecto descomprimido.

➤ Ejecutando el proyecto

Ejecutando el proyecto

Importación del proyecto en tu IDE

Descomprime el archivo ZIP descargado y abre el proyecto en tu Entorno de Desarrollo Integrado (IDE) favorito. La mayoría de los IDE populares, como IntelliJ IDEA, Eclipse o Visual Studio Code, son compatibles con proyectos de Spring Boot.

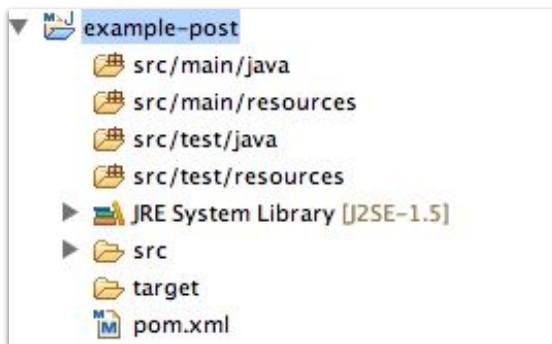
Ejecutando el proyecto

Estructura del proyecto

Un proyecto Spring Boot creado con Spring Initializr tendrá una estructura de directorios predefinida:

- **src/main/java** : Aquí se encuentra la ubicación principal para tus clases de Java.
- **src/main/resources** : Aquí se almacenan los recursos, como archivos de configuración.
- **src/main/webapp** (opcional): Si necesitas recursos web estáticos, como HTML, CSS o JavaScript, puedes ubicarlos aquí.
- **src/test** : Este directorio contiene las pruebas unitarias y de integración para tu aplicación.
- **pom.xml** (si usaste Maven): Estos archivos contienen la configuración del proyecto y las dependencias.

Ejecutando el proyecto



LIVE CODING

Creando un proyecto con Spring Initializr:

Vamos a poner en práctica lo aprendido! En este ejemplo vamos a crear un proyecto Spring Boot y abrirlo en nuestro IDE.

1- En primer lugar deberás acceder a la página <https://start.spring.io/>

Tiempo: 15 minutos

LIVE CODING

A continuación, genera las siguientes configuraciones:

1. Proyecto: Maven
2. Lenguaje: Java
3. Versión de Spring Boot: 3.1.2
4. Grupo: com.alke
5. Artefacto: wallet
5. Nombre: AlkeWallet
6. Descripción: Billetera Virtual
7. Paquete: com.alke.wallet
8. Empaquetamiento: WAR
9. Versión de Java: 8

LIVE CODING

A continuación, genera las siguientes configuraciones:

Luego, del lado derecho de la pantalla, deberás incluir las siguientes dependencias:

- Spring Web
- Spring Data JPA
- Spring Data JDBC
- MySQL Driver
- Thymeleaf

Finalmente descargar y descomprimir el archivo para poder abrir el proyecto en Eclipse.



Ejercicio N° 1

Primer proyecto Spring

Primer proyecto Spring

Contexto: 🙌

Ahora es tu turno de crear tu primer proyecto. Crea un proyecto Spring Boot en Spring Initializr con las siguientes particularidades:

Paso a paso: ⚙️

1. Proyecto Maven
2. Nombre de paquete raíz: com.ejercicio.spring
3. Version de Java: 8
4. Dependencias: Spring Web
5. Descargar .zip
6. Abrir en Eclipse.

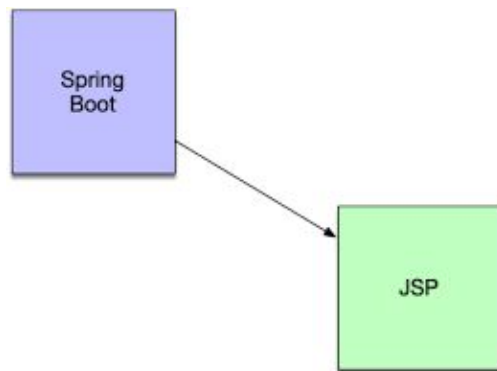
Tiempo 🕒: 15 minutos

» Configuración de Tecnologías de Vista

Configuración de la tecnología de vista

Spring Boot JSP

Las aplicaciones de Spring Boot JSP pueden ser más habituales de lo que en un primer momento nos parece . Sí bien es cierto que usar Spring Boot es usar tecnologías modernas que facilitan que despleguemos arquitecturas orientadas a MicroServicios, hay que recordar que siguen existiendo aplicaciones no tan “modernas” y quizás podamos necesitar que se ejecuten dentro de Spring Boot.



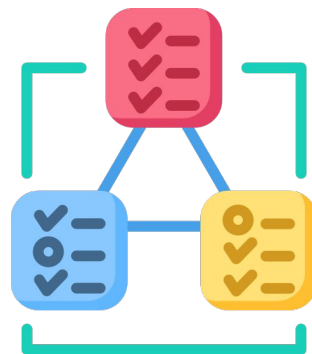
Configuración de la tecnología de vista

Spring Boot JSP

Con el proyecto generado en el punto anterior, ya tenemos la primera dependencia necesaria para un proyecto Spring Boot JSP, a continuación, deberíamos integrar también la siguiente dependencia (puede colocarse directamente en el pom.xml en el caso de que el proyecto ya esté generado).

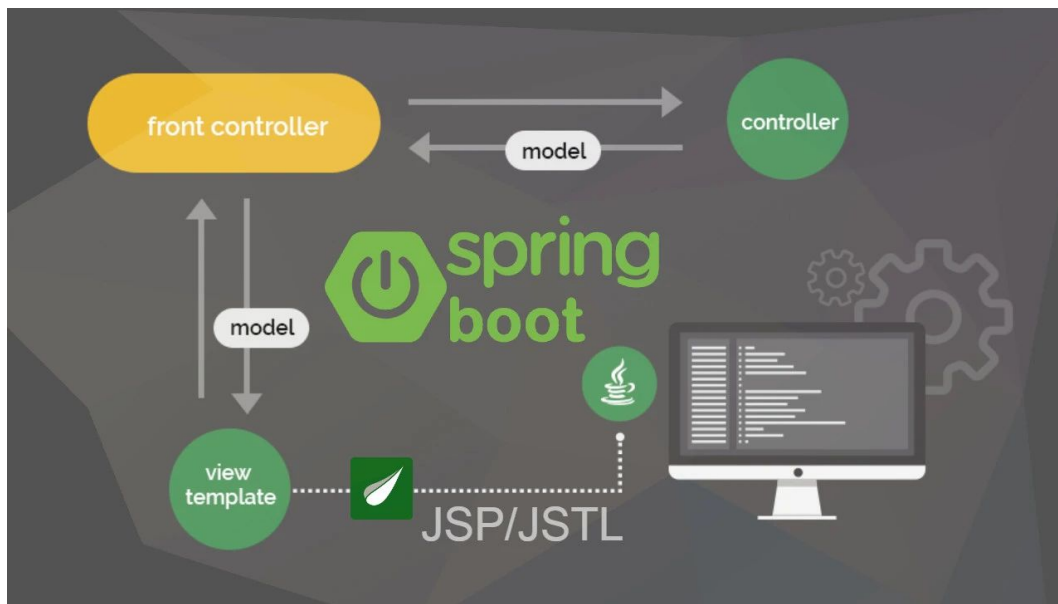
JSP requiere la dependencia del lenguaje JSTL, basado en etiquetas (Jakarta Standard Tag Library):

```
<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
  <version>3.0.0</version>
</dependency>
```



Configuración de la tecnología de vista

Spring Boot JSP

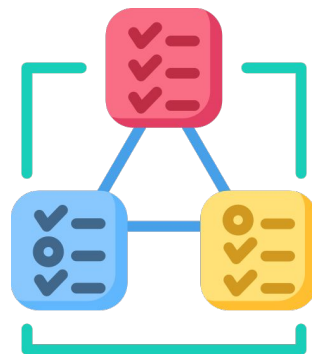


Configuración de la tecnología de vista

Spring Boot JSP

También precisa un motor de compilación porque cada fichero JSP se transforma en un servlet que será compilado. Los servlets son clases Java que procesan peticiones y respuestas HTTP.

```
<dependency>  
  <groupId>org.apache.tomcat.embed</groupId>  
  <artifactId>tomcat-embed-jasper</artifactId>  
  <scope>provided</scope>  
</dependency>
```



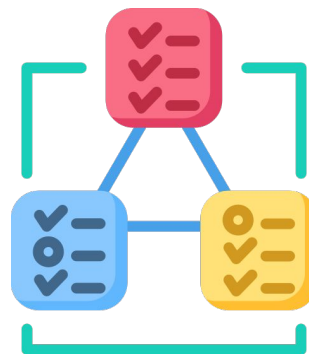
Configuración de la tecnología de vista

Spring Boot JSP

En el fichero de configuración `application.properties` definimos el prefijo y sufijo de los ficheros `.jsp`. El primero es la ruta en la que se ubican dentro de la carpeta `/src/main/webapp` y el segundo la extensión.

```
spring.mvc.view.prefix= /WEB-INF/views/  
spring.mvc.view.suffix= .jsp
```

De esta manera ya está completa la infraestructura inicial. Luego deberíamos escribir una clase especial denominada «controlador» con métodos que serán invocados cuando llamemos a ciertas direcciones vía HTTP.



Configuración de la tecnología de vista

Spring Boot Thymeleaf

Thymeleaf es una tecnología que permite definir una plantilla y, junto con un modelo de datos, obtener un nuevo documento. Es lo que se conoce como motor de plantillas.

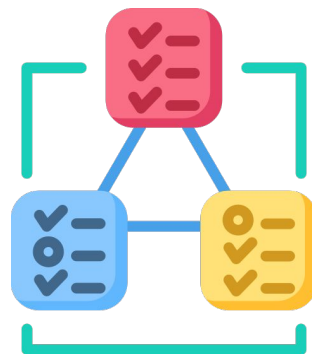


Configuración de la tecnología de vista

Spring Boot Thymeleaf

El motor de plantillas se genera para separar la interfaz de usuario (Vistas), de los datos comerciales (Modelos), pudiendo generar documentos en un formato específico, en este caso el motor de plantillas para el sitio web generará un documento HTML estándar.

Los motores de plantillas (templates engines) leen un fichero de texto que contiene la presentación ya preparada en HTML, e inserta en él la información dinámica que le ordena el Controlador, la parte que une la vista con la información.



Configuración de la tecnología de vista

Spring Boot Thymeleaf

La sintaxis a utilizar depende del motor de plantillas utilizado, pero todos son muy similares. Los motores de plantillas suelen tener un pequeño lenguaje de script que permite generar código dinámico, como listas o cierto comportamiento condicional, pero esto también depende del lenguaje.

Este lenguaje de script es absolutamente mínimo, lo justo para posibilitar ese comportamiento dinámico:

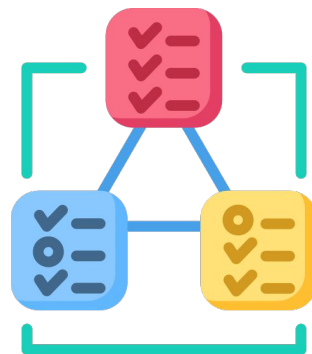
```
<html>
  <body>
    Hola ${nombre}
  </body>
</html>
```

Configuración de la tecnología de vista

Spring Boot Thymeleaf

Básicamente, el motor de plantillas se encarga de recibir una variable de tipo **String** llamada **nombre**, que luego se va a enviar desde el controlador a la vista y la hará parte del HTML, **haciéndolo dinámico**. Por lo que los diferentes usuarios verán diferentes resultados.

Thymeleaf permite realizar tareas que se conocen como natural templating. Es decir, cómo está basada en añadir atributos y etiquetas, sobre todo HTML, va a permitir que nuestras plantillas se rendericen en local, para que luego sean procesadas dentro del motor de plantillas (por lo cual las tareas de diseño y programación se pueden llevar a cabo conjuntamente).

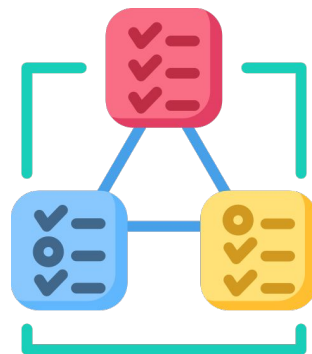


Configuración de la tecnología de vista

Spring Boot Thymeleaf

Permite trabajar con varios tipos de expresiones:

- **Expresiones variables** : Son quizás las más utilizadas y hacen referencia a una variable en particular, como por ejemplo `${variable}`
- **Expresiones de selección:** Son expresiones que nos permiten reducir la longitud de la expresión si pre fijamos un objeto mediante una expresión variable, como por ejemplo `*{objeto}`
- **Expresiones de enlace:** Nos permiten crear URL que pueden tener parámetros o variables, como por ejemplo `@{link}`



➤ Configurando un log en Spring MVC

< > Configurando un log en Spring MVC

Una vez que tenemos nuestra fantástica aplicación realizada con Spring lo normal es que queramos ver cómo se va comportando. Para ello, la manera más fácil es escribir mensajes dentro de la aplicación, explicando por cuáles funciones entra, como se toman las decisiones y, en general, como se va comportando.

Redirigiendo la salida

Spring Boot, como cualquier aplicación, por defecto mostrará todos los mensajes y/o errores por la salida estándar, pero a menudo deseamos guardar esos mensajes en un fichero. Para ello hay varias opciones.



< > Configurando un log en Spring MVC

La primera y más obvia es redirigir salida estándar y errores a un fichero. Así, en Linux y/o Windows, podríamos ejecutar nuestra aplicación de esta manera:

```
java -jar APP.JAR > PATH/FICHERO.LOG 2>&1
```

Esta manera funciona, pero tendremos que mirar en el fichero generado para ver como va todo, sin posibilidad de ver inmediatamente en nuestra consola como está funcionando el proceso recién lanzado.

Si lo que deseamos es que deje la salida estándar en un fichero, pero, que además, nos muestre en la consola esa misma salida, podremos lanzar el proceso de esta manera:

```
java -jar -Dlogging.file.name=PATH/FICHERO.LOG APP.JAR
```



< > Configurando un log en Spring MVC

Por supuesto también podemos poner en nuestro fichero `application.properties` de configuración esa misma propiedad y podremos lanzar el proceso sin el parámetro `'-D'`

`logging.file.name=PATH/FICHERO.LOG APP.JAR`

Si queremos ejecutar el proceso desde maven pondremos este comando:

`mvnspring-boot:run-Dspring-boot.run.arguments=-logging.file.name=PATH/FICHERO.LOG`



< > Configurando un log en Spring MVC

Configurando nivel de log

Por supuesto los mensajes que deseamos mostrar los podemos escribir con un simple `system.out.print`, pero desde luego no sería muy elegante ni práctico. Siempre es recomendable usar un `logger`.

Spring utiliza Commons Logging, usando por defecto Logback, si bien incluye también configuraciones por defecto para Java Util Logging y Log4J2.

Por defecto Spring Boot muestra los mensajes de nivel `ERROR-level`, `WARN-level`, y `INFO-level`, pero esto se puede cambiar de diferentes maneras.

La más simple es añadiendo el parámetro `--trace` o `--debug` cuando ejecutemos nuestro programa.

```
java -jar APP.JAR --debug
```



< > Configurando un log en Spring MVC

Configurando nivel de log

Otra manera será especificar en la propiedad '`logging.level.root`'. Esto lo haremos añadiendo la línea correspondiente en el fichero `application.properties`, o de otra manera, a la hora de ejecutarlo, añadiendo el parámetro adecuado:

```
java -jar -Dlogging.level.root=DEBUG APP.JAR
```

Una vez más si usamos maven para ejecutar el proceso, debemos escribir el siguiente comando:

```
mvn spring-boot:run -Dspring-boot.run.arguments =-logging.level.root=TRACE
```



< > Configurando un log en Spring MVC

Configurando nivel de log

Por supuesto, se permite especificar niveles de logging por paquetes, así, si queremos que nuestra aplicación que se encuentra en el paquete 'com.example.demo' muestre los mensajes de nivel debug, pero el resto de aplicación solo muestren los mensajes de error, escribiremos esta sentencia:

```
java -jar -Dlogging.level.root=ERROR  
-D-Dlogging.level.com.example.demo=DEBUG  
APP.JAR
```



< > Configurando un log en Spring MVC

Formateando la salida

Spring por defecto muestra los mensajes de error con este formator:

- Día y hora, incluyendo milisegundos.
- Nivel de Log: ERROR, WARN, INFO, DEBUG, o TRACE.
- Identificador o número del proceso
- Un --- como separador
- El nombre del Thread entre corchetes.
- El nombre de la clase donde se está escribiendo el log.
- El mensaje de log.

```
2019-03-05 10:57:51.112 INFO 45469 --- [main]
org.apache.catalina.core.StandardEngine : Starting Servlet
Engine: Apache Tomcat/7.0.52
2019-03-05 10:57:51.253 INFO 45469 --- [ost-startStop-1]
o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring
embedded WebApplicationContext
```



< > Configurando un log en Spring MVC

También podemos especificar que use colores para cuando muestre en la consola los logs, con la siguiente variable:

`spring.output.ansi.enabled=true`.

Así, los mensajes de error los mostrara en ROJO, los de aviso (WARN) en amarillo y los demás, en verde.

Configuración avanzada

Sí Spring Boot encuentra en el `classpath` un fichero con alguno de estos nombres, lo usará para configurar los loggers.

- logback-spring.xml
- logback.xml
- logback-spring.groovy
- logback.groovy



< > Configurando el DataSource

Spring Boot utiliza un algoritmo propio para buscar y configurar un DataSource. Esto nos permite obtener fácilmente una implementación de DataSource completamente configurada de forma predeterminada. Además, Spring Boot configura automáticamente un conjunto de conexiones ultrarrápidas, ya sea HikariCP, Apache Tomcat o Commons DBCP, en ese orden, dependiendo de cuáles están en la ruta de clases.

Si bien la configuración automática de DataSource de Spring Boot funciona muy bien en la mayoría de los casos, a veces necesitaremos un mayor nivel de control.



< > Configurando el DataSource

Lo primero que haremos será editar el fichero de configuración del proyecto (application.properties) para personalizarlo: application.properties

Configuración para el acceso a la Base de Datos

spring.jpa.hibernate.ddl-auto=update

spring.jpa.properties.hibernate.globally_quoted_identifiers=true

Puerto donde escucha el servidor una vez se inicie

server.port=8080

Datos de conexión con la base de datos MySQL

spring.datasource.url=jdbc:mysql://localhost:3306/myapp

spring.datasource.username=myappUser

spring.datasource.password=myappPassword



< > Configurando el DataSource

Hay que tener en cuenta que la propiedad `spring.jpa.hibernate.ddl-auto` se utiliza para que la base de datos se genere automáticamente en cada arranque de la aplicación. Esto nos interesará cuando estemos en desarrollo pero no cuando queramos desplegarla en producción. Por lo que tendremos que tener cuidado y controlar el valor de dicha propiedad.

- **none**: Para indicar que no queremos que genere la base de datos.
- **update**: Si queremos que la genere de nuevo en cada arranque.
- **create**: Si queremos que cree la DB pero que no la genere de nuevo si ya existe.



< > Ejecutando el proyecto

Estructura del proyecto

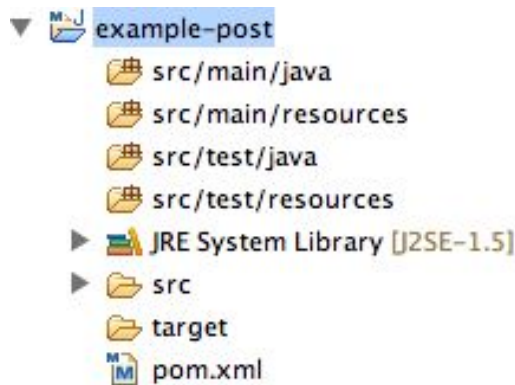
Un proyecto Spring Boot creado con Spring Initializr tendrá una estructura de directorios predefinida:

- **src/main/java** : Aquí se encuentra la ubicación principal para tus clases de Java.
- **src/main/resources** : Aquí se almacenan los recursos, como archivos de configuración.



< > Ejecutando el proyecto

- **src/main/webapp** (opcional): Si necesitas recursos web estáticos, como HTML, CSS o JavaScript, puedes ubicarlos aquí.
- **src/test**: Este directorio contiene las pruebas unitarias y de integración para tu aplicación.
- **pom.xml** (si usaste Maven): Estos archivos contienen la configuración del proyecto y las dependencias.



LIVE CODING

Creando una vista Spring Boot JSP:

Vamos a poner en práctica lo aprendido! En este ejemplo vamos a desplegar una vista JSP desde nuestro proyecto Spring Boot.

1. Crea un controlador que maneje las solicitudes y proporcione el mensaje que se mostrará en la página JSP. Puedes hacerlo creando una clase, como se muestra a continuación:

```
1 package cl.clases.yyy.controller;
2
3 import org.springframework.stereotype.Controller;
4 import org.springframework.ui.Model;
5 import org.springframework.web.bind.annotation.GetMapping;
6
7 @Controller
8 public class HolaController {
9
10     @GetMapping("/adios")
11     public String mostrarAdios(Model modelo) {
12
13         modelo.addAttribute("mensaje", "ADIOS! desde thymeleaf");
14         return "adios";
15     }
16 }
17
```

LIVE CODING

2. Crea un archivo JSP en el directorio :src/main/webapp/WEB-INF/views.
3. Crea un archivo llamado message.jsp con el siguiente contenido:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Mi aplicación SpringBoot JSP</title>
7 </head>
8 <body>
9   <h1>MENSAJE DESDE JSP</h1>
10  <p>${mensaje}</p>
11 </body>
12 </html>
13
```

Tiempo: 20 minutos

LIVE CODING

Finalmente, ejecuta la aplicación y accede desde tu localhost:

Abre un navegador web y navega a `http://localhost:8080/adios` (o al puerto que hayas configurado). Deberías ver la página JSP con el mensaje "ADIOS! desde thymeleaf".

Tiempo: 20 minutos

➤ Buenas prácticas de la industria (I) — Diseño y organización del proyecto

< > Garantizar que la aplicación web sea mantenible, escalable y fácil de comprender.

1. Estructura del proyecto clara

Organiza el código siguiendo el patrón MVC (Modelo–Vista–Controlador).

Cada capa debe tener una única responsabilidad:

- **Model:** contiene la lógica de negocio y las entidades.
- **View:** maneja la interfaz de usuario (HTML, JSP, Thymeleaf).
- **Controller:** gestiona las peticiones HTTP y conecta la vista con el modelo.

2. Convenciones de nombres y empaquetado

Usar nombres coherentes y descriptivos:

com.miapp.controller, com.miapp.service, com.miapp.model.

Esto facilita la lectura y el mantenimiento por otros desarrolladores.

3. Uso de dependencias controladas

Utilizar **Maven** o **Gradle** para manejar dependencias y versiones.

Evitar bibliotecas innecesarias que aumenten el tamaño o complejidad del proyecto.



➤ Buenas prácticas de la industria (II) — Desarrollo, seguridad y mantenimiento

< > **Garantizar que la aplicación web sea mantenible, escalable y fácil de comprender.**

4. Inyección de dependencias (IoC)

Aplicar la anotación `@Autowired` o constructor injection para evitar dependencias rígidas y facilitar el testing.

5. Configuración centralizada

Usar `application.properties` o `application.yml` para definir rutas, puertos y variables del entorno.

Ejemplo:

```
server.port=8080  
spring.datasource.url=jdbc:mysql://localhost:3306/miapp
```

6. Manejo de errores y excepciones

Implementar controladores globales con `@ControllerAdvice` y `@ExceptionHandler` para capturar errores y mostrar mensajes claros al usuario.

7. Seguridad y validaciones

Proteger rutas sensibles con Spring Security.

Validar datos de entrada usando anotaciones como `@NotNull`, `@Email` o `@Size`.



Momento:

Time-out!



5 -10 min.



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Conocer el framework Spring Boot y sus características
- ✓ Aprender sobre anotaciones
- ✓ Aprender sobre anotaciones e inyección de dependencias
- ✓ Conocer Spring Boot Initializr y sus características
- ✓ Aprender la estructura de un proyecto Spring
- ✓ Conocer la configuración de las tecnologías de vista
- ✓ Aprender a configurar un datasource



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

